面試模板

系統需求 (Requirements) (~ 5 minutes)

這個階段的目標，是清楚理解你被要求設計的是「什麼樣的」系統，不著急著畫架構圖。
為了避免一開始就迷失在細節中，建議你將「需求」分成兩個部分來思考：

1) 功能性需求（Functional Requirements）

功能性需求就是那些以「使用者 / 客戶應該能夠.....」開頭的敘述。

這些是系統的**核心功能**，應該是你與面試官討論的第一個主題。

在這個階段，你與面試官之間通常會有來回的互動討論。

你可以把自己想像成正在和 PM 或使用者討論需求，而不是在「背面試模板」。

同時，也可以開始從 **UI / UX 與使用流程** 的角度思考：

- 「哪些操作是核心功能、哪些是次要功能？」
- 「系統需要能做到 X 嗎？」
- 「如果發生 Y (e.g. 失敗、重試、重複請求)，系統應該怎麼處理？」

透過這樣的提問，你的目標是：

> 整理並「排序」出系統最核心的功能清單！

舉例來說，若你在設計一個類似 **Instagram** 的系統，可能的功能性需求包括：

- 使用者應該能夠發佈貼文（照片或影片）
- 使用者應該能夠追蹤其他使用者
- 使用者應該能夠看到他所追蹤對象的動態

又例如，如果你設計的是一個快取（cache）系統，需求可能會是：

- 客戶端應該能插入項目
- 客戶端應該能設定過期時間
- 客戶端應該能讀取項目

buildmoat.org

面試剩下的時間裡，你的主要任務是設計一個能滿足這些需求的系統。

因此，你需要在這個階段**有策略地挑選與排序重點功能**。

這些大型系統通常有上百個功能，但你的責任是：

> 請挑出你認為最重要的 3 個！

列出太長的需求清單反而會扣分 —— 許多 FAANG 面試會直接評估你「聚焦於重點的能力」。

2) 非功能性需求（Non-functional Requirements）

非功能性需求是描述系統品質的要求。

通常會以「系統應該能夠……」或「系統應該是……」的形式來表達。

舉例來說，如果你在設計一個類似 **Instagram** 的系統，非功能性需求可能包括：

- 系統應該具備高可用性（在一致性與可用性衝突時，以可用性為優先）
- 系統應該能擴展至支援超過 1 億 DAU（每日活躍使用者）
- 系統應該具備低延遲，能在 200 毫秒內載入使用者動態牆

非功能性需求應該放在具體情境中，並盡可能用可量化的方式表達。

例如：

- 「系統應該具備低延遲」太模糊，因為幾乎所有系統都希望低延遲。
- 「系統的搜尋延遲應低於 500ms」則更具意義，因為明確指出了哪個部分需要低延遲，並提供了具體目標。

若你對該領域不熟悉，想出非功能性需求會比較困難。

以下是一些幫助你思考的檢查清單。請嘗試找出與你的系統最相關的 3–5 個重點需求：

- **CAP 定理**：你的系統應優先一致性還是可用性？（e.g. 一致性（Consistence）、可用性（Availability）、分區容錯性（Partition Tolerance），在分散式系統中，容錯性是必然存在的）
- **環境限制**：系統運行的環境有無特殊條件？例如：

buildmoat.org

    - 手機裝置（電量有限）
    - 記憶體 / 頻寬受限的裝置（例如 3G 網路串流影片）
- **可擴展性（Scalability）**：是否有特殊的流量型態？
    - 是否在特定時間有突發流量？
    - 節日或熱門活動是否會造成巨量請求？
    - 讀寫比例如何？要擴展的是讀取還是寫入？
- **延遲（Latency）**：系統需要多快回應使用者？
    - 例如：在設計 Yelp 時，搜尋功能必須低延遲。
- **持久性（Durability）**：資料能否容忍遺失？
    - 社交網路可能能容忍部分遺失，但銀行系統不能。
- **安全性（Security）**：系統需多安全？
    - 涉及資料保護、權限控管、法規遵循。
- **容錯性（Fault Tolerance）**：
    - 系統在故障時的韌性如何？
    - 是否有冗餘、故障轉移與恢復機制？
- **合規性（Compliance）**：
    - 是否需符合特定法規或標準？
    - 如 GDPR、金融或醫療等產業規範。

buildnoat.org

面試時，不需要把所有非功能性需求都講完，而是：

> **只要挑出最重要的幾個，並說清楚「為什麼你會這樣選」就夠了。**

3) 容量估算（Capacity Estimation）

許多教學建議在這個階段進行「粗略估算（back-of-the-envelope calculation）」，但這通常沒有必要。除非這些計算會直接影響設計決策，否則可以略過。

在大多數情況下，你面對的是大型分散式系統，可以合理假設其規模龐大。

很多面試者會計算儲存空間、DAU、QPS，然後得出結論：

「OK，很多，了解。」

但這對面試官幾乎沒有幫助，只證明你會算數。

建議作法：

直接告訴面試官：「我會在設計過程中，若有需要時再進行估算。」，這樣更有效率。

什麼時候才需要計算？

舉例來說，若你設計 Instagram 熱門主題（Top K 熱門話題）系統，

你就需要估算：

- 系統每天可能處理多少主題
- 是否能用單一 min-heap 結構
- 還是需要分片（Sharding）分散處理

這些都會實際影響架構設計。

API 或系統介面（約 5 分鐘）

在進入高階架構設計（High-level Design）之前，你需要先定義系統與使用者之間的契約（contract）。

這個契約通常就是 API 介面，也就是你的系統要如何讓使用者或其他服務進行互動。在完整產品類型的系統設計面試中，這一步通常會直接對應到你前面整理出的功能性需求（但不一定完全相同）。你會利用這個契約來指導後續的高階設計，並確認你的系統確實能滿足那些需求。

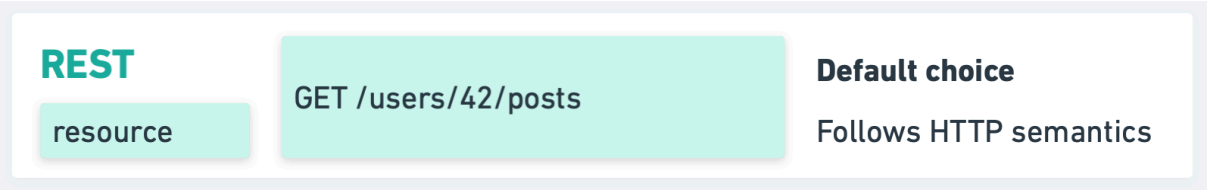
buildnoat.org

這裡你需要先快速決定，你**要使用哪一種 API 協定？**

以下是常見的三種選項：

1. REST（Representational State Transfer）

- 以 HTTP verbs（GET、POST、PUT、PATCH、DELETE）對資源進行 CRUD 操作。
- 是**大多數面試中預設的首選。**
- 資源導向、簡潔明確、可直觀對應至功能需求。

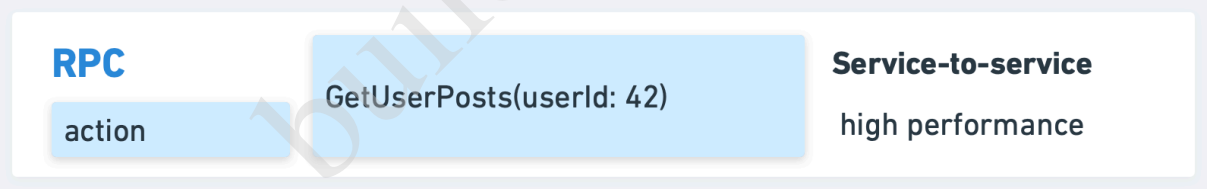


2. GraphQL

- 允許客戶端**精確指定想要取得的資料欄位**，避免 over-fetching（抓太多）或 under-fetching（抓太少）。
- 適合有多種前端客戶端（例如行動端、網頁端、合作方 API）且資料需求差異大的情境。

3. RPC（Remote Procedure Call）

- 以「動作」為導向的協定（例如 gRPC），在**服務對服務（service-to-service）通訊中通常比 REST 更快**。
- 適合內部服務間需要高效能通訊的情境。



buildmoat.org

實務建議

不用想太多，若沒有特別理由，**預設使用 REST** 就好。

若你的系統中包含即時互動功能（例如即時通知、直播、聊天室），可以在核心 API 設計完成後，再補上 WebSocket 或 Server-Sent Events（SSE）來支援即時傳輸。

以 Instagram 為例

- 使用者發佈一則貼文（含照片與文字說明）

```
POST /v1/posts
body: {
  "imageUrl": string,
  "caption": string
}
```

- 取得特定貼文內容

```
GET /v1/posts/{postId} -> Post
```

- 追蹤某位使用者（userId 由路徑辨識）

```
POST /v1/users/{userId}/follows
```

- 取得使用者動態牆（包含所追蹤帳號的貼文列表）

```
GET /v1/feed -> Post[]
```

設計注意事項

- 使用複數資源名稱（例如 `/posts` 而非 `/post`），以符合 REST 命名慣例。
- 用路徑參數表示資源關係（例如 `/users/{userId}/follows`），意思是「某個使用者（userId）的追蹤列表（follows）」。
- 避免在 request body 傳遞敏感資訊（如 userId），這些應從 authentication token 中取得。
- 所有請求都必須驗證身分，並從憑證推導出目前使用者，而非依賴使用者輸入。

buildmoat.org

大方向設計（約 10–15 分鐘）

到這一步，你已經明確掌握了系統需求（Requirements），以及 API 設計。現在你可以開始進行整體的大方向架構設計（High-Level Architecture）。

這個階段的重點，是畫出「方塊與箭頭圖」，也就是用簡單的圖示，表達系統中不同元件（components）以及它們之間的互動關係。

元件是什麼？

元件（Components）是系統的基本構成單位，例如：

- Web / App 伺服器
- 資料庫（Databases）
- 快取（Caches）
- 物件儲存（Object Storage）
- 訊息佇列（Message Queues）
- 負載平衡器（Load Balancer）

你可以在白板上手繪，也可以使用線上白板工具（例如 **Excalidraw**、**Whimsical**）繪製。

Tip: 在正式面試前，建議先詢問 recruiter 面試將使用哪一款工具，並事先練習操作。你絕對不希望在面試中浪費時間摸索畫圖軟體。

設計目標：滿足你的 API 與需求

不用過度思考或試圖一次做到完美。你的主要目標，是設計出一個能滿足先前 API 與需求的架構。在大多數情況下，你可以逐一檢視前面設計的每個 API endpoint，並依序構建出能支援它們的系統組件。這樣設計過程更自然，也能確保每個功能都被覆蓋到。

大方向比細節更重要

許多面試者在這一階段會犯的最大錯誤，就是「太快扣住細節」。結果導致時間用完，架構卻沒完成。

- 在大方向設計時，先專注在一個能滿足核心功能性需求的簡潔架構。
- 接著再在「深入探討（Deep Dive）」階段加入更進階的要素（e.g. 快取、分片、佇列、分散式一致性機制等），以滿足非功能性需求（如可擴展性、高可用性等）。

當你在大方向設計時發現某個部分有潛在可優化之處（e.g. 可以用快取加速、用佇列做解耦），只需要口頭指出「這裡之後可以優化」並在圖上做簡單註記即可，不用深入展開。

與面試官同步思考

在畫架構圖的同時，務必要邊講解你的思路。

buildnoat.org

明確說出每個步驟中的資料流與狀態變化：

- 從 API 請求進入系統開始
- 經過哪些層（如 Load Balancer → Web Server → Cache → Database）
- 每一層有哪些資料會被修改、儲存或傳遞
- 最終如何生成回應（Response）

這樣的講述能讓面試官清楚理解你對系統整體的掌握程度。

實體欄位的記錄方式

當請求流經資料庫或其他持久層（Persistence Layer）時，這是個好時機，可以開始標註各個核心實體（Entity）的關鍵欄位。

例如：

- 在 Instagram 中，你可以在資料庫方塊旁標註：
  - `User`: id, username, profile_pic_url
  - `Post`: id, user_id, image_url, caption, created_at
  - `Follow`: follower_id, followee_id

這樣能幫助你與面試官建立共同語境，也方便後續擴充。

你可以直接在圖上（例如資料庫方塊旁）標記欄位，讓資訊**靠近相關元件**、隨設計自然演化。

不必詳列所有欄位

不需要浪費時間列出明顯可推測的欄位。

例如，面試官都知道：

- `User` 表會有 name、email、password_hash
- `Post` 會有內容與建立時間

因此，請只專注標出對設計有意義、影響架構的欄位。

例如：

- 是否有 `is_public` 或 `visibility_scope` 欄位，影響快取或隱私處理？
- 是否需要 `like_count` 或 `comment_count` 來支援熱門排序？
- 是否需要 `thumbnail_url` 以便前端快速載入？

這些才是與設計決策相關、值得指出的細節。

buildmoat.org

深入探討（Deep Dives，約 10 分鐘）

深入探討的目的

在這個階段，你已經有了一個整體架構雛形，

接下來的 10 分鐘，目標是讓你的設計更「實戰化」與「可擴展」，具體來說要：

1. 確保設計能符合非功能性需求（例如高可用、可擴展、低延遲）。
2. 處理極端情況 Edge Cases。
3. 找出瓶頸與潛在問題，並提出改進方案。
4. 根據面試官的提示（probe）來主動調整設計。

主動性與資深程度的關聯

這個階段的「主導程度」會根據你的資歷而不同：

- 初階或中階工程師：面試官通常會主動指出哪裡可以改進，你的任務是能理解問題、提出合理解法並討論取捨。
- 資深或 Staff 級工程師：預期你能主動辨識問題並引導討論方向。例如主動說出：「在這個設計中，feed 讀取效能可能是瓶頸，我們可以考慮 fanout-on-write。」這能展現你對系統瓶頸的洞察力與架構思維。

實例：Instagram Feed 的深入優化

舉例來說，我們在最初的非功能性需求中提到，系統需要支援超過 1 億 DAU（每日活躍使用者）。

那麼這裡的深入討論就可以從以下方向展開：

- 如何水平擴展（Horizontal Scaling）應對龐大流量
- 如何引入快取（Cache）降低資料庫壓力
- 如何透過資料分片（Sharding）提升寫入與查詢效能
- 使用者的動態牆（feed）必須低延遲呈現：這是 Instagram（或其他社交平台）最有挑戰性的部分之一，因此這裡的深入探討可以圍繞：
    - **Fanout-on-write vs Fanout-on-read** 的策略比較：
        - Fanout-on-write：在使用者發佈貼文時，就主動推送到所有粉絲的 feed。

buildpat.org

        - Fanout-on-read：在讀取時即時組合所有追蹤對象的貼文。
    - 快取機制設計：
        - 使用記憶體型快取系統 (Redis or Memcached) 快取熱門 feed 或用戶首頁。
        - 討論 TTL（Time-to-Live）、快取失效策略、資料一致性等細節。

在這個階段，你的目標是帶領面試官一起優化架構，從最初的大方向設計架構，逐步進化成能應付真實規模的系統。

討論中的節奏與互動

有一個常見的錯誤：

面試者在這階段「講太多」，反而掩蓋了面試官想要聽到的內容。

確實，這階段的話題非常多，而且對資深工程師來說，主動性很重要。但也要記得保持平衡。

建議策略：

- 請主動引出話題（例如：「我想討論 feed 生成延遲的優化策略。」）
- 說明你的想法後，給面試官留出空間發問或深入探討。
- 記得，面試官往往有特定的「評估信號」要觀察，如果你講太多、沒有留白，他可能無法看到你想展現的優點。

此外，若你過度搶話，也會在「溝通與合作能力」項目上失分。

面試的目的不只是技術正確，更是在展示你的合作與系統性思考能力。

buildroot.org